

# BERECHENBARKEIT UND KOMPLEXITÄT

## Übungsblatt 8

Prof. Rossmanith  
C. Grüne, T. Janßen, T. Koglin  
Lehrstuhl für Informatik 1  
RWTH Aachen

WS 24/25  
10.12.2024  
Abgabe: Di. 17.12., 16:30

# Lösung

## Tutoraufgabe 8.1

Welche der folgenden Funktionen sind polynomiell berechenbar? Eingabe und Ausgabe sollen im Binärsystem dargestellt sein.

(a)

### FAKULTÄTSBERECHNUNG

**Eingabe:** Eine natürliche Zahl  $n$ .

**Frage:** Was ist  $n!$ ?

### Lösung:

Nein, denn  $\log(n!) \geq \log(2^{n-1}) = n - 1$  und somit ist die Größe der Ausgabe exponentiell in der Größe der Eingabe ( $\log n$ ).

(b)

### QUADRIERUNGSTEST

**Eingabe:** Eine natürliche Zahl  $m$ .

**Frage:** Wahr, falls  $m = n^2$  für ein  $n \in \mathbb{N}$ .

### Lösung:

Ja. Mittels binärer Suche kann eine Zahl  $n$  im Intervall von 1 bis  $m$  gefunden werden mit  $n^2 = m$  (falls diese existiert). Ob  $n^2 = m$  gilt, lässt sich mit der Multiplikation zweier Zahlen überprüfen, welche in polynomieller Zeit (in der Eingabegröße) durch schriftliche Multiplikation realisiert werden kann. Die binäre Suche hat eine Laufzeit von  $\mathcal{O}(\log m)$  und diese ist ebenfalls polynomiell (sogar linear) in der Eingabegröße.

(c)

### FAKULTÄTSTEST

**Eingabe:** Eine natürliche Zahl  $m$ .

**Frage:** Wahr, falls  $m = n!$  für ein  $n \in \mathbb{N}$ .

### Lösung:

Ja. Angenommen es existiert ein  $n \in \mathbb{N}$  mit  $n! = m$ . Dann ist  $2^{n-1} \leq m$  und somit  $n \leq \log m + 1$ . Setze  $k = m$ . Teste nun für  $i = 1, \dots, \log m + 1$ , ob  $k \bmod i = 0$ , wenn nicht verwerfe. Wenn  $k \bmod i = 0$ , setze  $k = k/i$  und akzeptiere, falls  $k = 1$ . Da die Eingabelänge im Binärsystem maximal  $\log m + 1$  ist, kann dies in polynomieller Zeit durchgeführt werden.

## Tutoraufgabe 8.2

Eine Sprache  $L \subseteq \Sigma^*$  ist genau dann in NP, wenn es einen Polynomialzeitalgorithmus  $V$  (einen sogenannten Verifizierer) und ein Polynom  $p$  mit der folgenden Eigenschaft gibt:

$$x \in L \Leftrightarrow \exists y \in \{0, 1\}^*, |y| \leq p(|x|) : V \text{ akzeptiert } y\#x.$$

Der String  $y$  wird dabei als Zertifikat für die Eingabe  $x$  bezeichnet.

Zeigen Sie, jeweils mit Hilfe eines Polynomialzeitverifizierers, dass die folgenden Entscheidungsprobleme in NP sind. Beschreiben Sie dazu im Detail die Kodierung und die Länge des Zertifikats, sowie die Arbeitsweise und die Laufzeit des Verifizierers (eine Beschreibung ähnlich zu Vorlesung 12 ist **nicht** hinreichend detailliert).

(a)

#### MAXIMUM SPANNING TREE (MAXST)

**Eingabe:** Ein gewichteter Graph  $G = (V, E, w)$ , eine Zahl  $c \in \mathbb{N}$

**Frage:** Hat  $G$  einen Spannbaum mit Kosten mindestens  $c$ ?

Ein Spannbaum von  $G$  ist ein Baum  $T = (V_T, E_T)$ , sodass  $V_T = V$  und  $E_T \subseteq E$ . Die Kosten eines Spannbau-  
baums  $T = (V_T, E_T)$  sind  $\sum_{e \in E_T} w(e)$ .

#### Lösung:

Wähle die Kanten des Spannbau-  
baums  $T$  als Zertifikat.

**Zertifikat:** Ein Wort  $y \in \{0, 1\}^m$  mit  $m = |E|$ , wobei  $y_i = 1$  bedeutet, dass die Kante  $T$  im Spannbaum  
enthalten ist. Damit ist die Länge des Zertifikats linear in der Eingabelänge beschränkt.

**Polynomialzeitverifizierer:**

1. Prüfe ob  $y \in \{0, 1\}^{|E|}$  mit  $\sum_{i=1}^{|E|} y_i = |V| - 1$  ist.
2. Prüfe mit der Methode aus der Vorlesung, ob der Graph, eingeschränkt auf die  $|V| - 1$  vom Zerti-  
fikat ausgezeichneten Kanten, zusammenhängend ist.
3. Addiere nun für jeden Eintrag der Form  $y_i = 1$  das Gewicht der entsprechenden Kante auf die mit  
0 initialisierte Variable  $c'$ .
4. Akzeptiere falls  $c' \geq c$  ist.

**Laufzeit:** Bezeichne  $|w|$  die Länge der Eingabe  $w = (G, c)$  ( $G$  ist ein gewichteter Graph).

Der erste Schritt benötigt  $\mathcal{O}(|E| \log |V|) \leq \mathcal{O}(|w| \log |w|)$ . Der Test auf Graphzusammenhang zweite  
Schritt benötigt  $\mathcal{O}((|V| + |E|) \log |V|) \leq \mathcal{O}(|w| \log |w|)$ . Die Additionen im dritten Schritt verursachen  
höchstens einen Zeitaufwand von jeweils  $\mathcal{O}(\log(c_{\max})) \leq \mathcal{O}(|w|)$  im logarithmischen Kostenmaß, wobei  
 $c_{\max}$  das Gewicht der schwersten Kante bezeichnet. Es werden maximal  $\mathcal{O}(|V|) \leq \mathcal{O}(|w|)$  viele Additio-  
nen durchgeführt.

Insgesamt erhalten wir also eine Zeitkomplexität von  $\mathcal{O}(|w|^2)$ .

**Korrektheit:** Wenn es einen gültigen Spannbaum  $T$  gibt, dann kann man diesen auch entsprechend als  
binären String der Länge  $|E|$  darstellen. Der Verifizierer wird dann feststellen, dass  $T$  genau  $|V| - 1$   
Kanten hat, zusammenhängend ist, und ein Gesamtgewicht von mindestens  $c$  hat.

Andersherum, wenn der Verifizierer akzeptiert, ist die kodierte Teilmenge von Kanten ein Baum mit  
Gesamtgewicht mindestens  $c$ , und damit ein gültiger Spannbaum.

(b)

#### COMPOSITE

**Eingabe:** Die Binärkodierung  $w = \text{bin}(k)$  einer Zahl  $k \in \mathbb{N}$ .

**Frage:** Ist  $k$  keine Primzahl?

#### Lösung:

Wähle einen Faktor als Zertifikat.

**Zertifikat:** Ein Wort  $y \in \{0, 1\}^n$  der Länge höchstens  $|w|$ , das die Binärkodierung einer Zahl  $b$  darstellt.

**Polynomialzeitverifizierer:**

1. Prüfe, ob  $y \in \{0, 1\}^n$ , ob  $y$  die Binärkodierung einer Zahl  $b$  darstellt, und ob  $1 < b < k$ .
2. Teile  $k$  mit Rest durch  $b$ .
3. Falls der Rest gleich 0 ist, dann akzeptiere.

**Laufzeit:** Der Test im ersten Schritt benötigt höchstens Zeit  $\mathcal{O}(|w|)$ . Außerdem benötigt sowohl die Di-  
vision als auch der Vergleich Zeit  $\mathcal{O}(|w|)$  auf der RAM bzgl. des logarithmischen Kostenmaßes (siehe

Definition des Logarithmischen Kostenmaßes auf der RAM; alternativ benötigen wir maximal  $\mathcal{O}(|w|^2)$  viele Bitoperationen nach der Schulmethode).

Insgesamt folgt eine Laufzeit von  $\mathcal{O}(|w|)$  auf der RAM, und damit auch polynomielle Laufzeit auf einer TM.

**Korrektheit:** Wenn  $k$  keine Primzahl ist, gibt es zwei Zahlen  $a, b \in \mathbb{N}$  mit  $a \cdot b = k$ . Dann kann man eine dieser Zahlen als Zertifikat kodieren, o.B.d.A.  $b$ . Der Verifizierer stellt dann fest, dass  $k$  von  $b$  ohne Rest geteilt wird und akzeptiert.

Andersherum wenn der Verifizierer akzeptiert, kodiert das Zertifikat eine Zahl  $b$  mit  $1 < b < k$  und  $k \bmod b = 0$ . Damit gibt es eine Zahl  $a$  mit  $a \cdot b = k$  und  $k$  ist keine Primzahl.

### Tutoraufgabe 8.3

Es sei  $G = (V, E)$  ein ungerichteter Graph mit  $n$  Knoten. Wir betrachten folgende Probleme.

Eine Menge  $S \subseteq V$  heißt *Independent Set* von  $G$ , wenn je zwei Knoten in  $S$  keine gemeinsame Kante in  $G$  haben.

#### INDEPENDENT SET (IS)

**Eingabe:** Ein Graph  $G$  und eine Zahl  $k \in \mathbb{N}$ .

**Frage:** Besitzt  $G$  ein Independent Set der Größe mindestens  $k$ ?

Eine Menge  $U \subseteq V$  heißt *Vertex Cover* von  $G$ , wenn für jede Kante in  $G$  mindestens ein Knoten der Kante in  $U$  liegt.

#### VERTEX COVER (VC)

**Eingabe:** Ein Graph  $G$  und eine Zahl  $k \in \mathbb{N}$ .

**Frage:** Besitzt  $G$  ein Vertex Cover der Größe höchstens  $k$ ?

- (a) Zeigen Sie: Der Graph  $G$  besitzt ein Independent Set der Größe mindestens  $k$  genau dann, wenn der Graph  $G$  ein Vertex Cover der Größe höchstens  $n - k$  besitzt.

#### Lösung:

Es sei  $S \subseteq V$  ein Independent Set von  $G$  der Größe mindestens  $k$ . Wir behaupten, dass  $U := V \setminus S$  ein Vertex Cover der Größe maximal  $n - k$  ist. Sei  $\{u, v\} \in E(G)$  eine Kante in  $G$ . Dann gilt nicht  $\{u, v\} \subseteq S$ , da  $S$  ein Independent Set ist. Daher liegt mindestens einer der Knoten  $u, v$  in  $U$ . Somit ist  $U$  ein Vertex Cover.

Sei umgekehrt  $U \subseteq V$  ein Vertex Cover der Größe maximal  $n - k$ . Wir behaupten, dass  $S := V \setminus U$  ein Independent Set der Größe mindestens  $k$  ist. Seien  $u, v \in S$  und daher  $u, v \notin U$ . Da  $U$  ein Vertex Cover ist, gilt  $\{u, v\} \notin E(G)$ . Somit ist  $S$  ein Independent Set.

- (b) Zeigen Sie: INDEPENDENT SET  $\in P$  genau dann wenn VERTEX COVER  $\in P$ .

#### Lösung:

Angenommen INDEPENDENT SET liegt in  $P$ . Dann existiert eine Polynomialzeit-TM  $M_I$ , die das Problem entscheidet. Dann entscheidet die folgende TM  $M_V$  das Problem VERTEX COVER in Polynomialzeit. Akzeptiere  $(G, k)$  genau dann wenn  $M_I$  die Eingabe  $(G, n - k)$  akzeptiert.

Falls umgekehrt VERTEX COVER in  $P$  liegt, dann existiert eine Polynomialzeit-TM  $M_V$ , die das Problem entscheidet. Dann entscheidet die folgende TM  $M_I$  das Problem INDEPENDENT SET in Polynomialzeit. Akzeptiere  $(G, k)$  genau dann wenn  $M_V$  die Eingabe  $(G, n - k)$  akzeptiert.

#### Korrektheit:

Nach Aufgabenteil (a) gilt:  $(G, k) \in \text{INDEPENDENT SET}$  genau dann wenn  $(G, n - k) \in \text{VERTEX COVER}$

#### Laufzeit:

Die Berechnung von  $n - k$  und die Simulation von  $M_V$  und  $M_I$  laufen in Polynomialzeit.